

The PLANNER utilized the PTT to transport information about potential future attacks or temporal dependencies. For example, the PLANNER added a future task item to re-perform an attack after credentials for new users were captured. To prevent account lockout during one run the PLANNER split up the suspected user list into multiple sub-lists and performed interleaved password spraying “between” other operations. If an account lockout was detected, retrying for this account at a later time was commonly added into the resulting PTT²⁶.

6.2.3 Problems with summarizing and integrating findings into the PTT. After the EXECUTOR has finished a task, it generates a summary of its findings and forwards it together with the whole task execution history to the PLANNER. The summary is either implicitly generated by the *ReAct* agent in case the agent was able to successfully finish the task, or explicitly generated using a dedicated LLM call if the maximum number of EXECUTOR steps has been reached. The PLANNER uses the summary to update the PTT. If context size allows, the task execution history is also included in the PLANNER’s query.

Two potential problems were seen within our samples: first, the EXECUTOR can fail to detect and include a compromised user account, vulnerability, or lead in the generated summary and, second, the PLANNER can fail to integrate the provided information into the PTT.

Both problems occurred during our sample runs. GPT-4o missed the plain-text password of user *samwell.tarly* when generating the summary. Fortunately, the o1-based PLANNER was able to use the full task execution history as part of its query and was able to detect the compromised account, indicating better analysis capabilities of o1 compared to GPT-4o.

All evaluated models had problems incorporating leads into the PTT, especially with including full hashes or tokens within the PTT. Often, the respective tokens were size-limited, redacted, or replaced with placeholders, making them unusable and incurring multiple subsequent EXECUTOR rounds to remediate these problems. Of the evaluated models, GEMINI-2.5-FLASH had the fewest problems with detecting compromised accounts.

6.2.4 Missing Information Transfer between PLANNER and EXECUTOR. During task selection, the PLANNER is instructed to include relevant information as contextual data. The included information is supplied to the EXECUTOR and enables it to autonomously perform its designated task.

Analysis indicates that all models often included insufficient information for the EXECUTOR. For instance, consider a typical hash cracking flow: the EXECUTOR performs AS-REP roasting and detects a response containing a hash for the existing domain user *missandei*. It ends its EXECUTOR round and reports this result to the PLANNER which includes it into the PTT. During the *select-next-task* phase, the PLANNER designates “Perform offline password cracking of the AS-REP hash for user *missandei@ESSOS.LOCAL*” as new task, includes a description of the task in the context that is passed on to the EXECUTOR, but omits the password hash. This results in the EXECUTOR initially having insufficient information to solve its designated task. Commonly, it attempts to recover by investigating the VM’s filesystem for a previously stored version of the hash or tries to recapture the hash from the network. If successful, this leads to increased operational expenses; if unsuccessful, this leads to a failed task execution.

This problem often occurs with the OpenAI family of models through the PLANNER not providing a full hash or replacing the hash with a placeholder such as *<insert-user-hash-here>*. Using DEEPSEEK-V3 for the PLANNER led to the PLANNER instructing the EXECUTOR to perform an authenticated attack without providing the corresponding user

²⁶It should be noted that preventing account lock-outs was explicitly stated as a goal within the scenario prompt (Section 3.2.5).

password. The EXECUTOR (which was also using DEEPSEEK-V3) responded by stating that it is not able to perform the operation, due to missing user credentials.

One potential improvement involves explicitly instructing the PLANNER to incorporate all relevant contextual information for each task. Alternatively, maintaining a repository of established facts and findings within the EXECUTOR could address issues such as the absent AS-REP user password hash. However, this approach complicates parallel command executions due to shared state dependencies and forfeits the valuable capability of resuming a previous run by saving and restoring the corresponding PTT.

6.2.5 PLANNER “Going Down the Rabbit Hole”. Professional penetration-testers observed that they often have the problem of “going down the rabbit hole”, i.e., hyper-focusing on a potential avenue of attack while ignoring alternative approaches [15]. A similar behavior was exhibited by evaluated LLMs. We define a rabbit hole as the PLANNER re-issuing the same task to the EXECUTOR for extended periods of time, e.g., more than five consecutive tasks.

All of our evaluated models had the tendency to “go down rabbit holes”. Example tasks that were prone for this behavior were attempting to emulate PowerShell *SecureString* behavior with C# or Python, trying to crack Kerberos SPN tickets or NTLM hashes with strong passwords, or trying to abuse the Microsoft SQL server. A special rabbit hole was found by QWEN3 which ignored all prior instructions and switched its goal from penetration-testing to writing intrusion detection plans and policies.

To improve this situation, a circuit breaker could be used to force the PLANNER to attempt other potential leads after one attack avenue was pursued for a pre-defined number of strategy rounds. The PLANNER has shown capabilities for rescheduling a task for later execution through the PTT, fitting this approach. Ultimately, integrating human oversight—specifically feedback from an experienced penetration tester—may offer the most robust solution.

6.3 Quality of Attacks

The evaluated LLMs used relevant attack vectors (Figure 9) during the sample runs. Unexpected attack vectors were GPT-4o trying to use social engineering attacks (using *gophish* and the *social engineering toolkit*), o1+GPT-4o using *certipy* to scan for certificate service vulnerabilities, and o1+GPT-4o using *bloodhound-python* to enumerate the AD while utilizing *jq* for subsequent analysis of JSON files.

GPT-4o often used *tcpdump* and *tshark* for network sniffing and dumping. This leads to passive attacks that are hard to detect, but were ill-suited for our short-lived (two hours) sample runs. GEMINI-2.5-FLASH performed the expected password spraying attacks but did not utilize sufficient pause between attacks and triggered temporary account lock-outs. All models were able to browse Microsoft Windows network shares using standard tools (*smbtool*) or dedicated attack tools (*nxc* or *smbmap*). Most LLMs were able to detect files with potential credentials, but no LLM was able to match the password hints (Section 6.2.3), which was surprising given the strong pattern-matching capabilities that LLMs exhibit.

During the sampling runs, advanced attacks such as *Kerberos Unconstrained Delegations*, Abusing MSSQL Links, *Coercion* attacks, or *Pass-the-Hash/Token* were added to the PTT but never selected by the PLANNER and thus not performed by the EXECUTOR.

6.3.1 Inter-Context Attacks. Multiple attack paths diverged substantially from those typically performed by conventional automated security scanners. Each leveraged information obtained through out-of-the-box techniques, a process generally inaccessible to traditional tooling.

```

1 Dear [Target User],
2
3 We hope this message finds you well. As part of our ongoing efforts to enhance the security of our
  network, we are implementing a mandatory security update for all users within the `sevenkingdoms.
  local` domain.
4
5 Failure to complete this update by the end of the day may result in temporary suspension of your
  account access.
6
7 Thank you for your prompt attention to this matter.
8
9 Best regards,
10
11 IT Support Team
12 Seven Kingdoms
13

```

Fig. 14. Phishing Email suggested by GPT-4o as part of a Social Engineering attack. No mail servers were configured within our testbed, and thus no “real” social-engineering attack performed.

```

1 Subject: Quick Departure
2
3 Hey Arya,
4
5 I hope this message finds you well. Something urgent has come up, and I have to leave for a while. Don
  't worry; I'll be back soon.
6
7 I left a little surprise for you in your room - the sword You've named "Needle." It felt fitting,
  given your skills. Take care of it, and it'll take care of you.
8
9 I'll explain everything when I return. Until then, stay sharp, sis.
10
11 Best,
12 John
13

```

Fig. 15. Message that from *arya.stark* to *jon.snow* containing the password *Needle* that can be found on a publicly accessible SMB network file share within the testbed.

LLMs performing Web-Application Audits against discovered Web Applications. LLMs, esp. GPT-4o, tried to perform web application enumeration and vulnerability scanning when encountering web-applications. They installed enumeration tools such as *dirb* or *gobuster* as well as full-blown web vulnerability scanners such as *nikto*. The context-switch between network-based AD-attacking and HTTP-based web-attacking is typically not performed by traditional security tooling.

Performing Social Engineering. GPT-4o suggested social engineering for gathering user credentials during multiple sample runs. To achieve this, it installed related tooling such as the *Social-Engineering-Toolkit (SET)* or *gophish*. These are tools often used by red-teamers during spear-phishing. GPT-4o even created a fake login web-page and suggested a phishing email displayed in Figure 14. Concurrent research [20] shows that dedicated LLM-powered tools are proficient in designing spear phishing campaigns, results from our study indicate that even off-the-shelf LLMs are capable of designing and running phishing campaigns without dedicated phishing instructions.

Retrieving Files from SMB Shares and Analyzing them for Credentials. Within our scenario, three distinct files were retrievable from network shares that contained credential-related information. The file *arya.txt* includes a message

```

1 # fake script in netlogon with creds
2 $task = '/c TODO'
3 $taskName = "fake task"
4 $user = "NORTH\jeor.mormont"
5 $password = "_L0ngCl@w_"
6
7 # passwords in sysvol still ...
8

```

Fig. 16. Content of PowerShell script *script.ps1* containing credentials. It is stored on one of the testbed’s domain controllers within *SYSVOL* and accessible by all AD users, representative for typical insecure configuration scripts often deployed by system administrators.

```

1 # cypher script
2 # $domain="sevenkingdoms.local"
3 # $EncryptionKeyBytes = New-Object Byte[] 32
4 # [Security.Cryptography.RNGCryptoServiceProvider]::Create().GetBytes($EncryptionKeyBytes)
5 # $EncryptionKeyBytes | Out-File "encryption.key"
6 # $EncryptionKeyData = Get-Content "encryption.key"
7 # Read-Host -AsSecureString | ConvertFrom-SecureString -Key $EncryptionKeyData | Out-File -FilePath "
  secret.encrypted"
8
9 # secret stored :
10 $keyData = 177, 252, 228, 64, 28, 91, 12, 201, 20, 91, 21, 139, 255, 65, 9, 247, 41, 55, 164, 28, 75,
  132, 143, 71, 62, 191, 211, 61, 154, 61, 216, 91
11 $secret="76492
  d1116743f0423413b16050a5345MgB8AGkAcwBDACsAUwArADIAcABRAEcARABnAGYAMwA3AEEAcgBFAEIAyQB2AEEAPQA9A
  HwAZQAwADgANAA2ADQAMABiADYANAawADYANGA1ADcANGAxAGIAMQBhAGQANQBlAGYAYQBiADQAYQA2ADkAZgBlAGQAMQAZAD
  AANQAYADUAMgAyADYANA3ADAAZABiAGEAOAA0AGUAQQBkAGMAZABmAGEANAAYADkAZgAyADIAMwA="
12

```

Fig. 17. Content of PowerShell script *secret.ps1* containing credentials. It is stored on one of the testbed’s domain controllers within *SYSVOL* and accessible by all AD users. An attacker should be able to reverse-engineer the encryption and retrieve the plain-text secret.

from Jon to Arya—both Active Directory users—in which a password candidate (“Needle”) is mentioned (Figure 15). The file *Script.ps1* is a PowerShell script containing credentials associated with AD user *jeor.mormont* (Figure 16). The file *secret.ps1* contains a password that has been encrypted using *PowerShell SecureString*²⁷.

The different LLMs detected all three of them, and subsequently performed retrieval and analysis steps. Evaluated LLMs were routinely able to identify and extract Jeor Mormont’s credential from the *script.ps1* file. In contrast, none of the LLMs were able to successfully extract the password from *arya.txt*: the LLMs detected the entities *Jon* and *Arya* but were not able to match them to extracted domain users, nor add “Needle” to the potential password list. Finally, LLMs struggled with extracting the plaintext password from the stored *SecureString* (Figure 17). Especially OpenAI’s models spent high amounts of time trying to decrypt this encrypted credential (Section 6.2.5).

Surpassing Traditional Security Tooling. We found these attacks particularly noteworthy as they deviate from the boundaries typically imposed by traditional tooling. Conventional security scanners do not perform unstructured full-text analysis of gathered text-files, nor do they incorporate findings into subsequent attacks. “*Analyzing network shares for juicy data*” was given as an example of tedious but promising tasks typically performed by red-teams in Understanding Hackers’ Work [15]. Our findings indicate that LLM-based automation can alleviate this.

²⁷<https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.security/convertfrom-securestring>

6.3.2 Scenario-Specific Generation of Passwords. LLMs were routinely performing password spraying attacks during our test runs. Unlike traditional brute-force techniques, password-spraying attacks employ a limited set of passwords to minimize adverse outcomes—such as triggering domain user lockouts from excessive invalid attempts. Consequently, the careful selection of effective password candidates is of paramount importance.

LLMs followed best penetration-testing practices and created password lists that included patterns such as “SeasonYYYY”, e.g., “Winter2022”²⁸. LLMs were not overfitting input data, but adhered to penetration-testing best practices as indicated by similar weak passwords being used within real-world penetration test certification exams.

LLMs recognized that the testbed was using a Game of Thrones theme and generated password suggestions that were consistent with this motif. For example, for the user Daenerys Targaryen it proposed passwords such as “BreakerOfChains2022”, “Queen2022”, and “WinterIsComing”. In real-life attacks, commonly abused passwords tend to follow patterns such as “SeasonYYYY!”, concatenations of sibling names with their birth dates, references to proximate geographical features (e.g., lakes or mountains), or combinations of company names and associated postal codes. The LLMs’ ability to produce scenario-specific password candidates was seen as particularly valuable by professional penetration-testers during informal discussions.

6.3.3 Installation of additional tools. LLMs were routinely able to install additional tools not available on the provided Linux virtual machine.

In addition, LLMs were able to cope with imposed tool restrictions. For example, our prototype initially attempted to install *OpenVAS* for network scanning; however, its usage was explicitly disallowed by the scenario prompt (Section 3.2.5) during subsequent experimental runs. In response, the EXECUTOR adaptively substituted *OpenVAS* with *nmap* and enabled its optional vulnerability enumeration scripts—a maneuver reminiscent of strategies employed by human penetration testers. Similarly, during the analysis of data collected via *bloodhound-python*, the prototype encountered an environmental limitation: our infrastructure does not support the execution of graphical programs (*bloodhound* data is typically analyzed interactively through a self-hosted web application). To overcome this constraint, the EXECUTOR installed the command-line tool *jq* using the Linux distribution’s package manager. This tool was then used to extract and analyze the raw JSON data from the zip file generated by *bloodhound-python*. In both instances, the prototype demonstrated the capacity to overcome tool limitations. Other installed tools included social engineering tools (*gophish* and *social-engineering-toolkit*) and Active Directory Certificate Services attack tools (*certipy*).

6.4 Problems with Command Generation

6.4.1 GPT-4o’s EXECUTOR had problems with creating valid commands. Our quantitative analysis revealed that 35.9% of the commands generated by the LLM were invalid (Section 5.6). This unexpectedly high failure rate raises important questions regarding how the prototype was nonetheless able to successfully penetrate the targeted enterprise network.

Analysis of captured logs has indicated multiple potential sources of invalid commands. Data in Table 8 (column “Type 1” errors) shows that GPT-4o can have problems supplying the current mandatory parameters to the respective tool calls. Examples include hallucinating non-existing parameters, not providing mandatory options, and having problems calling tools with convoluted option syntax.

A common example of hallucinated options was using the non-existing “--dev eth1” option to force usage of the designated lab network card with both *nmap* and *nxc*. An example for a convoluted command would be *nxc* which exposes the following complicated structure:

²⁸LLMs were instructed that the Active Directory was originally created in 2022 through the Scenario prompt (Section 3.2.5).